Overview of Spring Boot

Spring Boot makes it easy to create stand-alone, production-grade Spring based Applications that you can "just run".

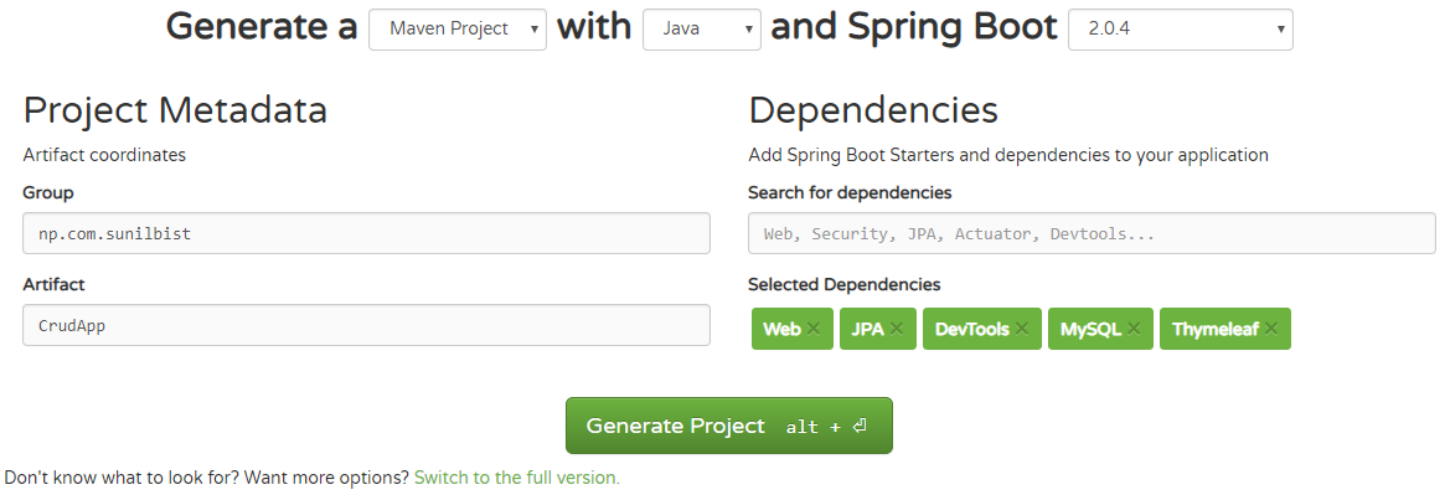
We take an opinionated view of the Spring platform and third-party libraries so you can get started with minimum fuss. Most Spring Boot applications need very little Spring configuration.

Features

- Create stand-alone Spring applications
- Embed Tomcat, Jetty or Undertow directly (no need to deploy WAR files)
- Provide opinionated 'starter' dependencies to simplify your build configuration
- Automatically configure Spring and 3rd party libraries whenever possible
- Provide production-ready features such as metrics, health checks and externalized configuration
- Absolutely no code generation and no requirement for XML configuration

Bootstrap your application with Spring Initializers

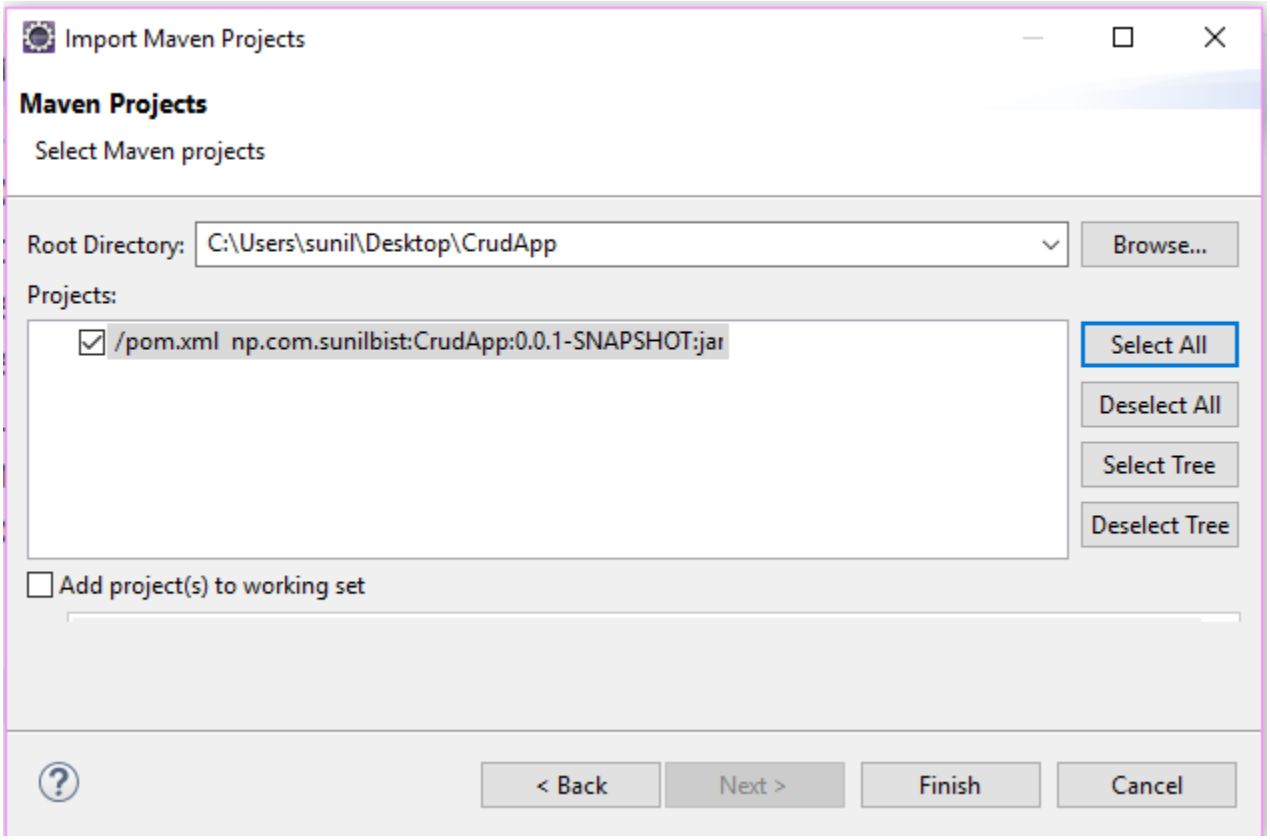
Step 1: Go to https://start.spring.io/ and you will see a screen and make changes as shown below



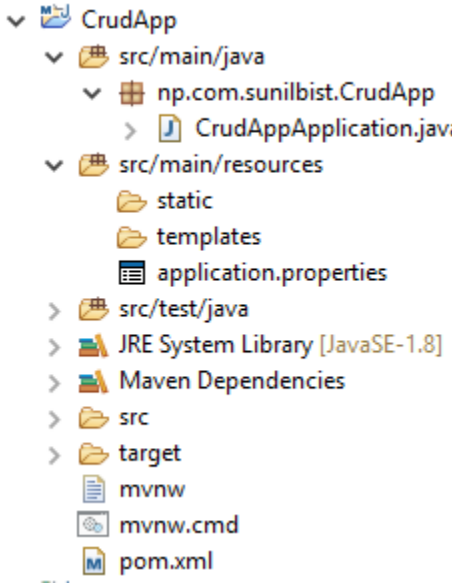
Step 2: Download and Extract to your desired location for my case I am extracting to desktop

Step 3: Now open your JavaEE eclipse version and import maven existing project as below.

Note: - This will require internet connection to your computer because it will download all the required libraries from maven repository from the remote server to your local computer. Once all the required libraries are downloaded next time you do not need to connect for the same jar file. It will search from your .m2 folder under your user name folder.



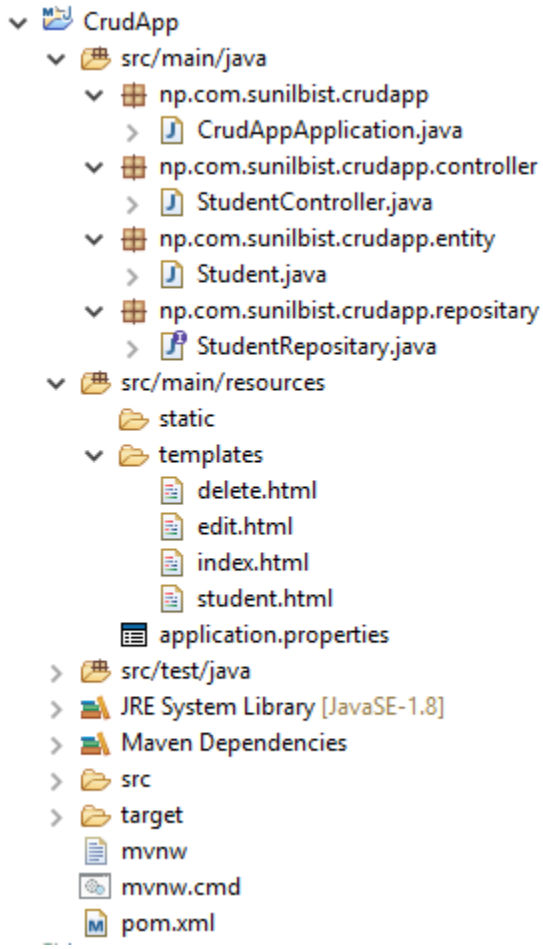
When down load completes its folder, structure looks like below screen



Step 4: - Now Setup your database and hibernate properties in the ***application.properties*** file as below: -

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/crudapp
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto = update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL5Di
alect
spring.thymeleaf.cache=false
```

Change your package name all in lowercase and create various packages as shown in figure below



Student.java

```
package np.com.sunilbist.crudapp.entity;
import javax.persistence.Entity;
```

```
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;

@Entity
public class Student {

	@Id
	@GeneratedValue (strategy=GenerationType.AUTO)
	private int id;

	private String name;

	public Student() {
	}

	public void setId(int id) {
		this.id = id;
	}
	public int getId() {
		return id;
	}
	public void setName(String name) {
		this.name = name;
	}
	public String getName() {
		return name;
	}
}
```

StudentRepositary.java

```
package np.com.sunilbist.crudapp.repositary;
import org.springframework.data.repository.CrudRepository;
import org.springframework.stereotype.Repository;
import np.com.sunilbist.crudapp.entity.Student;

@Repository
public interface StudentRepositary extends CrudRepository<Student,
Integer> {

}
```

StudentController.java

```java
package np.com.sunilbist.crudapp.controller;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import np.com.sunilbist.crudapp.entity.Student;
import np.com.sunilbist.crudapp.repositary.StudentRepositary;


@Controller
@RequestMapping("/student")
public class StudentController {

	@Autowired
	private StudentRepositary studentRepo;

	@GetMapping("/list")
	public String list(Model model){
		model.addAttribute("studentList",studentRepo.findAll());
		return "index";
	}

	@GetMapping("/add")
	public String add(Model model){
		model.addAttribute("title", "New Student");
		model.addAttribute("student", new Student());
		return "student";
	}

	@PostMapping("/add")
	public String add(@ModelAttribute("student") Student student){
		studentRepo.save(student);
		return "redirect:/student/list";
	}

	@GetMapping("/edit/{id}")
	public String edit(@PathVariable("id") Integer id, Model model){
		model.addAttribute("student", studentRepo.findById(id));
		return "edit";
	}

	@PostMapping("/edit")
```

```
	public String edit(@ModelAttribute("student") Student student){
		studentRepo.save(student);
		return "redirect:/student/list";
	}

	@GetMapping("/delete/{id}")
	public String delete(Model model, @PathVariable("id") Integer
id){
		model.addAttribute("student", studentRepo.findById(id));
		return "delete";
	}

	@PostMapping("/delete")
	public String delete(@ModelAttribute("student") Student student){
		studentRepo.delete(student);
		return "redirect:/student/list";
	}
}
```

CrudAppApplication.java

```
package np.com.sunilbist.crudapp;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class CrudAppApplication {

	public static void main(String[] args) {
		SpringApplication.run(CrudAppApplication.class, args);
	}
}
```

index.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="ISO-8859-1">
<title>Student List</title>
</head>
<body>
	<a th:href="@{/student/add}">Add New</a>
	<table border="1">
		<thead>
```

```
                <tr>
                    <th>#</th>
                    <th>Name</th>
                    <th>Action?</th>
                </tr>
            </thead>
            <tbody>
                <tr th:each="s : ${studentList}">
                    <td th:text="${s.id}"></td>
                    <td th:text="${s.name}"></td>
                    <td><a
th:href="@{'/student/delete/'+${s.id}}">Delete</a> <a

      th:href="@{'/student/edit/'+${s.id}}">Edit</a></td>
                </tr>
            </tbody>
            <tfoot>
                <tr>
                    <th>#</th>
                    <th>Name</th>
                    <th>Action?</th>
                </tr>
            </tfoot>
      </table>
</body>
</html>
```

student.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="ISO-8859-1">
<title th:text="${title}">Student</title>
</head>
<body>
      <h3 th:text="${title}"></h3>
      <hr />
      <form action="#" method="post" th:action="@{/student/add}"
            th:object="${student}">
            <input type="text" th:field="*{name}" />
            <button type="submit">Save</button>
      </form>
</body>
</html>
```

edit.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="ISO-8859-1">
<title th:text="${title}">Edit</title>
</head>
<body>
      <h3 th:text="${title}"></h3>
      <hr />
      <form action="#" method="post" th:action="@{/student/edit}"
            th:object="${student}">
            <input type="hidden" th:field="*{id}" />
            <input type="text" th:field="*{name}" />
            <button type="submit">Update</button>
      </form>
</body>
</html>
```

delete.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="ISO-8859-1">
<title th:text="${title}">Delete</title>
</head>
<body>
      <h3 th:text="${title}"></h3>
      <hr />
      <form action="#" method="post" th:action="@{/student/delete}"
            th:object="${student}">
            <input type="hidden"  th:field="*{id}" /> <input
type="hidden"
                  th:field="*{name}" />
            <h3>Are you sure you want to delete?</h3>
            <button type="submit">Yes</button>
            <a th:href="@{/student/list}">No</a>
      </form>
</body>
</html>
```

Important note: - Don't forget to create database to your mysql database crudapp and update application.properties file based on your mysql database credentials

Now its time to run and test your CrudApp

Right click on **CrudAppApplicaiton java** file and Run as Java Application and you it works everything is good to go

Best wishes for your coming future By Sunil Bahadur Bist

Professional Java Developer and Instructor www.sunilbist.com.np
info@sunilbist.com.np